OpenMP — Lesson 10

Finding data races with ThreadSanitizer

Gustavo Lozano

November 2026

Source: Chapman, Jost & van der Pas [1], Ch. 7 "Troubleshooting" (data races and tools to detect them). Tool documentation: the ThreadSanitizer and Archer pages of the LLVM project.

1 The idea

Lesson 2 showed that a data race gives wrong results *sometimes*: with small `N` the program usually looked correct. Testing the output is therefore not enough to know that a parallel program is correct. We need a tool that watches the memory accesses themselves.

ThreadSanitizer (TSan) is built into GCC and Clang. With `-fsanitize=thread` the compiler adds a check to every memory access. While the program runs, TSan records which thread touched which address and which synchronization (barriers, locks, ...) happened in between. When two threads access the same address, at least one writes, and no synchronization orders them, TSan prints a report with the file and line of both accesses.

2 The catch: the tool must understand OpenMP

TSan knows the synchronization of `pthreads` (mutexes, thread creation), but OpenMP runtimes implement barriers and reductions with their own internal tricks. If TSan cannot see them, it reports races that do not exist (*false positives*).

- **GCC** (`libgomp`): TSan does not see OpenMP's synchronization, so it reports false races even in correct programs.
- **Clang** (`libomp`) has **Archer**, a small library that tells TSan what OpenMP synchronizes. Load it with the environment variable `OMP_TOOL_LIBRARIES`.

So for this lesson we use **Clang + Archer**. On Arch Linux:

```
sudo pacman -S clang openmp compiler-rt
find /usr/lib -name libarcher.so        # check that Archer is installed
```

3 The code

The buggy counter from Lesson 2, with a smaller `N` because TSan makes the program 5–15 times slower.

`tsan_counter.c`

```
/*
 * tsan_counter.c -- OpenMP Lesson 10: finding a data race with ThreadSanitizer.
 * Source: Chapman, Jost & van der Pas (2007), "Using OpenMP",
 *         Ch. 7 (Troubleshooting: data races, tools to find them).
 *
```

```c
 * This is the buggy counter from Lesson 2, made smaller (the tool slows
 * the program down 5-15x).
 *
 * Arch Linux packages:  sudo pacman -S clang openmp compiler-rt
 *
 * Compile with ThreadSanitizer (-g gives file and line numbers in the report):
 *     clang -g -fopenmp -fsanitize=thread -o tsan_counter tsan_counter.c
 * Run, loading Archer (the helper that teaches the tool OpenMP's rules):
 *     export TSAN_OPTIONS='ignore_noninstrumented_modules=1'
 *     export OMP_TOOL_LIBRARIES=$(find /usr/lib -name libarcher.so | head -1)
 *     OMP_NUM_THREADS=2 ./tsan_counter
 */

#include <stdio.h>
#include <omp.h>

#define N 100000

int main(void)
{
    int counter = 0;              /* shared by default (Lesson 2) */

    #pragma omp parallel          /* EXPERIMENT step 2: add  reduction(+:counter)  here */
    {
        for (int i = 0; i < N; i++)
            counter++;            /* DATA RACE: the tool should point at THIS line */
    }

    printf("expected = %d, counter = %d\n", omp_get_max_threads() * N, counter);
    return 0;
}
```

```
clang -g -fopenmp -fsanitize=thread -o tsan_counter tsan_counter.c
export TSAN_OPTIONS='ignore_noninstrumented_modules=1'
export OMP_TOOL_LIBRARIES=$(find /usr/lib -name libarcher.so | head -1)
OMP_NUM_THREADS=2 ./tsan_counter
```

Output of a real run on Arch Linux (Clang/LLVM 23, `/usr/lib/libarcher.so`):

```
(env) [glozanoac@bell 10-openmp-thread-sanitizer]$ clang -g -fopenmp -fsanitize=thread -o tsan_counter tsan_counter.c
(env) [glozanoac@bell 10-openmp-thread-sanitizer]$ export TSAN_OPTIONS='ignore_noninstrumented_modules=1'
(env) [glozanoac@bell 10-openmp-thread-sanitizer]$ find /usr/lib -name libarcher.so | head -1
/usr/lib/libarcher.so
(env) [glozanoac@bell 10-openmp-thread-sanitizer]$ export OMP_TOOL_LIBRARIES=$(find /usr/lib -name libarcher.so | head -1)
(env) [glozanoac@bell 10-openmp-thread-sanitizer]$ OMP_NUM_THREADS=2 ./tsan_counter
==================
WARNING: ThreadSanitizer: data race (pid=152533)
  Write of size 4 at 0x7ffcbeae1af4 by thread T1:
    #0 main.omp_outlined_debug__ /home/glozanoac/projects/self/studies/scientific-programming/10-openmp-thread-sanitizer/tsan_
counter.c:31:20 (tsan_counter+0xed69d) (BuildId: 361bc0cdff2582e2143fa1fc07a8a22db4caad71)
    #1 main.omp_outlined /home/glozanoac/projects/self/studies/scientific-programming/10-openmp-thread-sanitizer/tsan_counter.
c:28:5 (tsan_counter+0xed705) (BuildId: 361bc0cdff2582e2143fa1fc07a8a22db4caad71)
    #2 __kmp_invoke_microtask <null> (libomp.so+0xc94c8) (BuildId: a646cf85deab87c593a4a6b8595ead2e4b899fb1)
    #3 main /home/glozanoac/projects/self/studies/scientific-programming/10-openmp-thread-sanitizer/tsan_counter.c:28:5 (tsan_
counter+0xed5e9) (BuildId: 361bc0cdff2582e2143fa1fc07a8a22db4caad71)

  Previous write of size 4 at 0x7ffcbeae1af4 by main thread:
    #0 main.omp_outlined_debug__ /home/glozanoac/projects/self/studies/scientific-programming/10-openmp-thread-sanitizer/tsan_
counter.c:31:20 (tsan_counter+0xed69d) (BuildId: 361bc0cdff2582e2143fa1fc07a8a22db4caad71)
    #1 main.omp_outlined /home/glozanoac/projects/self/studies/scientific-programming/10-openmp-thread-sanitizer/tsan_counter.
c:28:5 (tsan_counter+0xed705) (BuildId: 361bc0cdff2582e2143fa1fc07a8a22db4caad71)
    #2 __kmp_invoke_microtask <null> (libomp.so+0xc94c8) (BuildId: a646cf85deab87c593a4a6b8595ead2e4b899fb1)
    #3 main /home/glozanoac/projects/self/studies/scientific-programming/10-openmp-thread-sanitizer/tsan_counter.c:28:5 (tsan_
counter+0xed5e9) (BuildId: 361bc0cdff2582e2143fa1fc07a8a22db4caad71)

  Location is stack of main thread.
```

```
  Location is stack of main thread.

  Thread T1 (tid=152535, running) created by main thread at:
    #0 pthread_create <null> (tsan_counter+0x6728e) (BuildId: 361bc0cdff2582e2143fa1fc07a8a22db4caad71)
    #1 __kmp_create_worker /usr/src/debug/llvm/llvm-project-23.1.4.src/openmp/runtime/src/z_Linux_util.cpp:866:7 (libomp.so+0x
a456a) (BuildId: a646cf85deab87c593a4a6b8595ead2e4b899fb1)

SUMMARY: ThreadSanitizer: data race /home/glozanoac/projects/self/studies/scientific-programming/10-openmp-thread-sanitizer/ts
an_counter.c:31:20 in main.omp_outlined_debug__
==================
expected = 200000, counter = 117981
ThreadSanitizer: reported 1 warnings
(env) [glozanoac@bell 10-openmp-thread-sanitizer]$
```

Note the last lines: in this run the counter is wrong (`117981` instead of `200000`), and TSan reported 1 warning.

4 Reading the report

Report line	Meaning
`Write of size 4 ...by thread T1`	thread T1 wrote 4 bytes (an `int`)
`#0 ...tsan_counter.c:31:20`	where: line 31, column 20, the `counter++`
`Previous write ...by main thread`	the conflicting earlier access, by another thread, with no synchronization in between
`Location is stack of main thread`	the variable is a local of `main`: `counter`, shared by default
`#1 main.omp_outlined`	`omp_outlined` is the function the compiler made from the parallel region (*outlining*, Chapman Ch. 8)

Key ideas

- TSan points to the exact line and says which threads collided. You no longer guess from wrong numbers.
- TSan reports the race even in runs where the counter happens to come out correct (200000). It detects that a race *can* corrupt the result, not only that it did.
- Always compile with `-g`, otherwise the report has no file and line numbers.
- Use TSan only for testing: it slows the program down and uses much more memory.

5 The experiment

Experiment

Question: can the tool tell a racy program from a correct one?

1. Compile and run `tsan_counter.c` as above (Clang + Archer). Read the report.
2. Fix the race: change the pragma to `#pragma omp parallel reduction(+:counter)`. Recompile with Clang and run again, with the same environment variables.
3. Compile the *fixed* program with GCC instead, and run it:

```
gcc -g -fopenmp -fsanitize=thread -o tsan_gcc tsan_counter.c
OMP_NUM_THREADS=2 ./tsan_gcc
```

What to observe and answer.

1. In step 1, which line does the report point to? Was the printed counter wrong?
2. In step 2 there is no report. Does that *prove* the program has no race?
3. In step 3, GCC's TSan reports a race in the fixed program. Is it real? Why does it appear?

Step 2 on Arch Linux (the fixed program saved as `tsan_counter_fix.c`): the counter is correct and TSan prints no warning.

```
(env) [glozanoac@bell 10-openmp-thread-sanitizer]$ clang -g -fopenmp -fsanitize=thread -o tsan_counter_fix tsan_counter_fix.c
(env) [glozanoac@bell 10-openmp-thread-sanitizer]$ OMP_NUM_THREADS=2 ./tsan_counter_fix
expected = 200000, counter = 200000
(env) [glozanoac@bell 10-openmp-thread-sanitizer]$
```

Answers (read after trying). (1) Line 31, `counter++`, written by two threads. The counter may well be correct: TSan makes the program slow, so the threads interleave less, which hides the wrong result but not the race. The tool reports the unsynchronized accesses, whatever the result. (2) No. TSan is a *dynamic* tool: it only checks the accesses that actually happen in this run, with this input and these threads. A race in code that this run never executed is not reported. Silence means "no race seen", which is strong evidence but not proof. (3) It is a false positive. `libgomp` combines the private copies of the reduction with its own internal synchronization, which TSan does not see, so it looks like an unprotected write. Lesson: a race detector must understand the threading library. For OpenMP, use Clang with Archer, or a dedicated tool such as Intel Inspector.

Next lesson. This closes the Chapman book. Next we move to van der Pas et al. [2]: tasks (`task` and `taskwait`), for parallel work that is not a loop.

References

[1] B. Chapman, G. Jost, and R. van der Pas. *Using OpenMP: Portable Shared Memory Parallel Programming*. MIT Press, Cambridge, MA, 2007.

[2] R. van der Pas, E. Stotzer, and C. Terboven. *Using OpenMP — The Next Step: Affinity, Accelerators, Tasking, and SIMD*. MIT Press, Cambridge, MA, 2017.